

Obstacle Avoidance Robot

TECHNICAL PROJECT DOCUMENTATION

1. Project Overview

An Obstacle Avoidance Robot is a fully autonomous vehicle equipped with environmental awareness capabilities. Unlike a line follower that strictly adheres to a predefined path, this robot explores its environment and dynamically alters its trajectory to prevent collisions with physical barriers. It utilizes ultrasonic sound waves to continuously map the distance to objects ahead of it.

Primary Objectives:

- To build an autonomous roaming robot using an Arduino microcontroller.
- To integrate and interface an HC-SR04 Ultrasonic Distance Sensor.
- To program a real-time decision-making algorithm that triggers evasive maneuvers when an obstacle breaches a safe distance threshold.

2. Hardware Components Required

Component	Quantity	Specification / Purpose
Arduino UNO	1	Main processing unit (ATmega328P) that executes the avoidance logic.
HC-SR04 Ultrasonic Sensor	1	Acts as the "eyes" of the robot, emitting and receiving sound waves.
L298N Motor Driver	1	H-Bridge module to provide sufficient current to the DC motors.
Geared DC Motors (BO)	2	Actuators that drive the left and right wheels independently.
Robot Chassis & Wheels	1 Set	Acrylic/wood body, 2 drive wheels, and 1 caster wheel for balance.
Power Supply	1	9V to 12V Battery pack (Lithium-ion recommended).

Why Ultrasonic? IR sensors struggle with black or transparent objects and ambient sunlight. Ultrasonic sensors use sound waves, making them immune to lighting conditions and object color, providing highly reliable distance measurements.

3. Working Principle (SONAR Logic)

The HC-SR04 sensor operates on the principle of echolocation, similar to bats or submarines. It consists of two main cylindrical modules: a Transmitter and a Receiver.

1. The Arduino triggers the sensor to emit an ultrasonic sound pulse (inaudible to humans) at 40 kHz.
2. The sound wave travels through the air until it hits an object and bounces back.
3. The Receiver catches the reflected wave (echo).
4. The sensor outputs the time duration it took for the wave to travel out and back.

Distance Calculation Formula:

The speed of sound in air is approximately 340 meters per second, or 0.034 cm per microsecond (µs). Since the sound wave travels to the object AND back, we divide the total time by 2.

Distance (cm) = (Duration in µs × 0.034) / 2

4. Circuit Connections & Pin Mapping

Accurate wiring is crucial for the safety of the microcontroller and the correct functioning of the algorithms.

Module	Module Pin	Connected To (Arduino / Power)
HC-SR04 Ultrasonic Sensor	VCC	Arduino 5V
	GND	Arduino GND
	TRIG (Trigger)	Arduino Analog Pin A0 (Used as digital out)
	ECHO (Receiver)	Arduino Analog Pin A1 (Used as digital in)
L298N Motor Driver	12V Power In	Positive (+) terminal of 12V Battery
	GND	Battery Negative (-) AND Arduino GND
	5V Out	Arduino VIN (Provides power to the Arduino)
	IN1, IN2, IN3, IN4	Arduino Digital Pins 8, 9, 10, 11 (Motor control)

Note: Analog pins A0 and A1 on the Arduino can function perfectly as Digital Input/Output pins. They are used here to keep the digital rail (pins 0-13) less cluttered.

5. Software Implementation (C++ Code)

The code controls the robot's movement based on distance thresholds. The safe distance is set to **20 cm**. If an object is detected within this range, the robot halts, reverses, and pivots to find a clear path.

Part 1: Setup and Distance Calculation

```
/* * Project: Obstacle Avoidance Robot * Description: Uses HC-SR04 to avoid collisions. */ // Define Sensor Pins const int trigPin = A0; const int echoPin = A1; // Define Motor Driver Pins const int in1 = 8; const int in2 = 9; const int in3 = 10; const int in4 = 11; // Variables for calculating distance long duration; int distance; void setup() { // Configure motor driver pins pinMode(in1, OUTPUT); pinMode(in2, OUTPUT); pinMode(in3, OUTPUT); pinMode(in4, OUTPUT); // Configure Ultrasonic sensor pins pinMode(trigPin, OUTPUT); pinMode(echoPin, INPUT); } void loop() { // 1. Send out an ultrasonic pulse digitalWrite(trigPin, LOW); delayMicroseconds(2); digitalWrite(trigPin, HIGH); delayMicroseconds(10); digitalWrite(trigPin, LOW); // 2. Read the echo return time duration = pulseIn(echoPin, HIGH); // 3. Calculate distance in cm distance = duration * 0.034 / 2; // 4. Decision Making if (distance < 20 && distance > 0) { // Obstacle detected within 20cm stopRobot(); delay(300); // Pause moveBackward(); delay(400); // Reverse for 0.4 seconds turnRight(); delay(500); // Turn right for 0.5 seconds } else { // Path is clear moveForward(); } }
```

Part 2: Motor Control Functions

```
// --- Movement Routines --- void moveForward() { // Both motors rotate forward digitalWrite(in1, HIGH); digitalWrite(in2, LOW); digitalWrite(in3, LOW); digitalWrite(in4, HIGH); } void moveBackward() { // Both motors rotate in reverse digitalWrite(in1, LOW); digitalWrite(in2, HIGH); digitalWrite(in3, HIGH); digitalWrite(in4, LOW); } void turnRight() { // Right motor stops, Left motor goes forward digitalWrite(in1, LOW); digitalWrite(in2, LOW); digitalWrite(in3, LOW); digitalWrite(in4, HIGH); } void stopRobot() { // Cut power to all motors digitalWrite(in1, LOW); digitalWrite(in2, LOW); digitalWrite(in3, LOW); digitalWrite(in4, LOW); }
```

6. Calibration & Troubleshooting

- **Robot spins in circles constantly:** Check the `pulseIn()` function and ensure the TRIG and ECHO pins are not physically swapped on the Arduino. If the sensor returns a faulty reading (e.g., 0 cm), the robot will assume it is constantly blocked.
- **Motors lack power / Arduino resets:** Ensure the 12V battery is fully charged. DC motors draw a spike in current when reversing direction. If the battery is weak, the voltage drop will cause the Arduino to restart repeatedly.
- **Blind Spots:** The HC-SR04 has a detection cone of roughly 15-20 degrees. It may miss very thin obstacles (like table legs) or sound-absorbing materials (like thick carpets/sponges).

Advanced Upgrade Path: To eliminate blind spots, you can mount the HC-SR04 on a **Micro Servo Motor (SG90)**. Before turning, the robot can stop, sweep the sensor Left and Right, compare the distances, and autonomously decide which direction has more open space.